High-Concurrency Movie Ticket Booking

Database: MySQL 8.0.16+ with InnoDB **Time convention:** store `DATETIME(6)` show/hold timestamps in UTC; convert to the theatre's IANA timezone for display.

Scenario and behavior

For a theatre, show today and the next six theatre-local dates. After a date is selected, list scheduled movies, screens, local show start/end times, and currently available seat counts. Seats are held transactionally; payment webhook events have a unique provider/event key so duplicate deliveries do not repeat booking effects.

Files and execution

• `sql/schema.sql`: creates the database, all tables, indexes, constraints, and triggers.

• `sql/seed.sql`: inserts the example theatre, screen, physical seats, movies, shows, customer, and per-show inventory.

• `sql/queries.sql`: executable P2 query plus MySQL hold, expiry, and webhook query patterns.

• `submission/High-Concurrency-Booking-Database.pdf`: formatted submission document.

Run with a MySQL 8.0.16+ client:

```
mysql -u root -p < sql/schema.sql
mysql -u root -p < sql/seed.sql
mysql -u root -p < sql/queries.sql
```

The P2 query uses named session variables at the top of `sql/queries.sql`. Change `@theatre_id` and `@local_date` for another request. MySQL named time zones (such as `Asia/Kolkata`) must be loaded into the MySQL time-zone tables for `CONVERT_TZ` to return values.

Entities and attributes

Table	Attributes and keys	Purpose
`movies`	`movie_id` CHAR(36) PK; `title` VARCHAR(200); `language` VARCHAR(60); `duration_mins` SMALLINT UNSIGNED; `rating` VARCHAR(20) nullable; `created_at` DATETIME(6)	Movie catalogue.
`theatres`	`theatre_id` CHAR(36) PK; `name` VARCHAR(160); `address` VARCHAR(500); `timezone` VARCHAR(64)	Venue and theatre-local date/time conversion.
`screens`	`screen_id` CHAR(36) PK; `theatre_id` CHAR(36) FK; `name` VARCHAR(80); UNIQUE (`theatre_id`, `name`)	Theatre auditorium.
`seats`	`seat_id` CHAR(36) PK; `screen_id` CHAR(36) FK; `row_label` VARCHAR(8); `seat_number` SMALLINT UNSIGNED; `seat_type` ENUM; UNIQUE (`screen_id`, `row_label`, `seat_number`)	Physical seating plan.
`shows`	`show_id` CHAR(36) PK; `movie_id` CHAR(36) FK; `screen_id` CHAR(36) FK; `starts_at`, `ends_at` DATETIME(6) UTC; `status` ENUM; `currency` CHAR(3); UNIQUE (`screen_id`, `starts_at`)	One movie screening on one screen. Triggers serialize schedule writes on the screen and reject overlapping scheduled intervals.
`customers`	`customer_id` CHAR(36) PK; `display_name` VARCHAR(160); `email` VARCHAR(254) UNIQUE; `created_at` DATETIME(6)	Booking customer.
`bookings`	`booking_id` CHAR(36) PK; `customer_id` CHAR(36) FK; `show_id` CHAR(36) FK; `status` ENUM; `created_at`, `expires_at` DATETIME(6)	Hold and booking lifecycle. Amount is derived from seat price snapshots; currency is on the show.
`show_seats`	(`show_id`, `seat_id`) composite PK and FKs; `price_amount` DECIMAL(12,2); `state` ENUM; `hold_expires_at` DATETIME(6) nullable	One row per seat per show and the seat-locking boundary. A trigger verifies the seat belongs to the show's screen.
`booking_seats`	(`booking_id`, `seat_id`) composite PK; both FKs; `price_amount` DECIMAL(12,2)	Booking-to-seat association and historical price snapshot. A trigger verifies the seat has inventory for the booking's show.
`payments`	`payment_id` CHAR(36) PK; `booking_id` FK; `provider`; nullable `provider_payment_id`; `idempotency_key`; `amount`; `status`; `created_at`, `updated_at`; UNIQUE (`provider`, `idempotency_key`) and (`provider`, `provider_payment_id`)	Payment attempt and provider idempotency identifiers.
`payment_webhook_events`	(`provider`, `provider_event_id`) composite PK; nullable `payment_id` FK; `received_at`, `processed_at`; `payload` JSON	Durable webhook event deduplication and audit record.

Sample rows

The seed script uses stable IDs and is safe to rerun. Its example records include:

Table	Example
`theatres`	PVR Central, MG Road, Bengaluru, `Asia/Kolkata`
`screens`	Screen 1 at PVR Central
`seats`	A1/A2 standard seats and B1/B2 premium seats
`movies`	The Last Monsoon (Kannada, 142 min); Orbit Seven (English, 128 min)
`shows`	The Last Monsoon at 09:00 local and Orbit Seven at 12:30 local on 2026-10-15; another screening on 2026-10-16
`customers`	Sample Customer (`sample.customer@example.test`)
`show_seats`	First show's A1 is `BOOKED` at INR 250.00; other standard/premium seats are available at INR 250.00/350.00

Table	Example
`bookings`	One confirmed booking for the first show, created 2026-10-14 10:00 UTC
`booking_seats`	That booking owns seat A1 at the INR 250.00 historical price
`payments`	One successful demo payment for INR 250.00 (`demo-payment-001`)
`payment_webhook_events`	One processed `payment.succeeded` event (`evt-demo-001`)

`sql/seed.sql` contains the complete executable INSERT statements and IDs.

Normalization

• **1NF:** Every column holds one scalar. A booking's seats are rows in `booking_seats`, never a packed list.

• **2NF:** Single-column-key tables satisfy 2NF. In `show_seats`, price/state/expiry depend on the full `(show_id, seat_id)` key. In `booking_seats`, the price snapshot depends on `(booking_id, seat_id)`.

• **3NF:** Movie, theatre, screen, seat, and customer descriptions are stored only with their own entities. Booking currency is not copied from its show, and booking total is calculated from the immutable seat price snapshots.

• **BCNF:** Declared determinants are candidate keys or superkeys. Examples include `screens`: `screen_id` and `(theatre_id, name)`; `seats`: `seat_id` and `(screen_id, row_label, seat_number)`; `shows`: `show_id` and `(screen_id, starts_at)`; `show_seats`: `(show_id, seat_id)`; `booking_seats`: `(booking_id, seat_id)`; `payments`: `(provider, idempotency_key)` and `(provider, provider_payment_id)` when non-null; webhook events: `(provider, provider_event_id)`. Other base entities are keyed by their primary key.

Concurrency and lifecycle

For a hold, start a transaction and select the requested `show_seats` rows in ascending `seat_id` order using `FOR UPDATE SKIP LOCKED`. Continue only if every unique requested seat was returned and is `AVAILABLE`; otherwise roll back. Insert the `HELD` booking and `booking_seats` price snapshots, update the locked inventory rows to `HELD` with one database-generated expiry, then commit. This makes multi-seat holds all-or-nothing. The composite inventory primary key means only one inventory row exists for each show/seat pair, and its row lock serializes competing buyers.

An expiry worker claims due `HELD` bookings in small batches using `FOR UPDATE SKIP LOCKED`; in one transaction it locks their seat rows in ascending order, changes the booking to `EXPIRED`, and returns the inventory to `AVAILABLE`. An unswept expired hold stays unavailable briefly rather than being double-sold.

Webhook handling inserts `(provider, provider_event_id)` before applying effects, in the same transaction. A duplicate-key error (MySQL error 1062) means the event was already applied; acknowledge it without repeating effects. Lock the booking before its seats and lock seats in ascending order. Confirm only a still-live hold, updating payment, booking, and inventory atomically. If the hold expired first, persist the payment result and trigger a refund/compensation workflow; never confirm seats that were released. External payment/refund calls occur outside database transactions.

MySQL has no PostgreSQL-style exclusion constraint; the show triggers lock the screen row and reject overlapping scheduled shows. All schedule writes must use the same trigger-protected path. Keep transactions short and retry deadlocks with bounded backoff.

P1 SQL

The complete MySQL table creation and sample INSERT statements are in `sql/schema.sql` and `sql/seed.sql`. Hold, expiry, and webhook SQL patterns are in `sql/queries.sql`.

P2 SQL

This query is executable in MySQL 8.0 after running the schema and seed files. It uses the theatre-local date, handles daylight-saving day boundaries via `CONVERT_TZ`, returns local show timings, and filters to the next seven local dates.

```
SELECT
    sh.show_id,
    m.title AS movie_title,
    sc.name AS screen_name,
    CONVERT_TZ(sh.starts_at, '+00:00', th.timezone) AS starts_at_local,
    CONVERT_TZ(sh.ends_at, '+00:00', th.timezone) AS ends_at_local,
    sh.currency,
    SUM(CASE WHEN ss.state = 'AVAILABLE' THEN 1 ELSE 0 END) AS available_seats
FROM theatres AS th
JOIN screens AS sc ON sc.theatre_id = th.theatre_id
JOIN shows AS sh ON sh.screen_id = sc.screen_id
JOIN movies AS m ON m.movie_id = sh.movie_id
LEFT JOIN show_seats AS ss ON ss.show_id = sh.show_id
WHERE th.theatre_id = @theatre_id
  AND sh.status = 'SCHEDULED'
  AND @local_date >= DATE(CONVERT_TZ(UTC_TIMESTAMP(), '+00:00', th.timezone))
AND @local_date < DATE_ADD(DATE(CONVERT_TZ(UTC_TIMESTAMP(), '+00:00', th.timezone)), INTERVAL 7
      DAY)
  AND sh.starts_at >= CONVERT_TZ(CONCAT(@local_date, ' 00:00:00'), th.timezone, '+00:00')
AND sh.starts_at < CONVERT_TZ(CONCAT(DATE_ADD(@local_date, INTERVAL 1 DAY), ' 00:00:00'),
      th.timezone, '+00:00')
GROUP BY sh.show_id, m.title, sc.name, sh.starts_at, sh.ends_at, th.timezone, sh.currency
ORDER BY sh.starts_at, sc.name;
```

Submission notes

The referenced theatre screenshot was not available as a workspace file, so the SQL and document use the described date-picker/showtime flow rather than reproducing the image. Git is available through the installed GitHub Desktop bundle, but this checkout has no configured remote; a PR URL cannot be created until the target GitHub repository is supplied and authenticated.

The MySQL concurrency harness is `tests/load_test.py`. With the schema applied to a disposable MySQL database, install `requirements.txt`, set `MYSQL_HOST`, `MYSQL_USER`, and `MYSQL_PASSWORD`, then run `python tests/load_test.py --attempts 1000 --workers 64`. It asserts single-seat exclusivity, all-or-nothing two-seat holds, expiry release, webhook deduplication, and an expiry/payment race outcome.

Appendix A: MySQL Schema (P1)

```
-- MySQL 8.0.16+. All DATETIME(6) values are UTC by application convention.
CREATE DATABASE IF NOT EXISTS booking_engine
    CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;
USE booking_engine;

CREATE TABLE movies (
    movie_id CHAR(36) NOT NULL PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    language VARCHAR(60) NOT NULL,
    duration_mins SMALLINT UNSIGNED NOT NULL,
    rating VARCHAR(20) NULL,
    created_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    CONSTRAINT chk_movies_duration CHECK (duration_mins > 0)
) ENGINE=InnoDB;

CREATE TABLE theatres (
    theatre_id CHAR(36) NOT NULL PRIMARY KEY,
    name VARCHAR(160) NOT NULL,
    address VARCHAR(500) NOT NULL,
    timezone VARCHAR(64) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE screens (
    screen_id CHAR(36) NOT NULL PRIMARY KEY,
    theatre_id CHAR(36) NOT NULL,
    name VARCHAR(80) NOT NULL,
    CONSTRAINT uq_screens_theatre_name UNIQUE (theatre_id, name),
    CONSTRAINT uq_screens_id_theatre UNIQUE (screen_id, theatre_id),
    CONSTRAINT fk_screens_theatre FOREIGN KEY (theatre_id)
        REFERENCES theatres (theatre_id)
) ENGINE=InnoDB;

CREATE TABLE seats (
    seat_id CHAR(36) NOT NULL PRIMARY KEY,
    screen_id CHAR(36) NOT NULL,
    row_label VARCHAR(8) NOT NULL,
    seat_number SMALLINT UNSIGNED NOT NULL,
    seat_type ENUM('STANDARD', 'PREMIUM', 'ACCESSIBLE') NOT NULL,
    CONSTRAINT uq_seats_position UNIQUE (screen_id, row_label, seat_number),
    CONSTRAINT uq_seats_id_screen UNIQUE (seat_id, screen_id),
    CONSTRAINT chk_seats_number CHECK (seat_number > 0),
    CONSTRAINT fk_seats_screen FOREIGN KEY (screen_id)
        REFERENCES screens (screen_id)
) ENGINE=InnoDB;

CREATE TABLE shows (
    show_id CHAR(36) NOT NULL PRIMARY KEY,
    movie_id CHAR(36) NOT NULL,
    screen_id CHAR(36) NOT NULL,
    starts_at DATETIME(6) NOT NULL COMMENT 'UTC',
    ends_at DATETIME(6) NOT NULL COMMENT 'UTC',
    status ENUM('SCHEDULED', 'CANCELLED') NOT NULL DEFAULT 'SCHEDULED',
    currency CHAR(3) CHARACTER SET ascii NOT NULL,
    CONSTRAINT uq_shows_screen_start UNIQUE (screen_id, starts_at),
    CONSTRAINT uq_shows_id_screen UNIQUE (show_id, screen_id),
    CONSTRAINT chk_shows_time CHECK (ends_at > starts_at),
    CONSTRAINT chk_shows_currency CHECK (currency REGEXP '^[A-Z]{3}$'),
    CONSTRAINT fk_shows_movie FOREIGN KEY (movie_id) REFERENCES movies (movie_id),
    CONSTRAINT fk_shows_screen FOREIGN KEY (screen_id) REFERENCES screens (screen_id),
    INDEX ix_shows_movie_start (movie_id, starts_at),
    INDEX ix_shows_screen_status_start (screen_id, status, starts_at)
) ENGINE=InnoDB;

CREATE TABLE customers (
    customer_id CHAR(36) NOT NULL PRIMARY KEY,
    display_name VARCHAR(160) NOT NULL,
    email VARCHAR(254) NOT NULL,
    created_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    CONSTRAINT uq_customers_email UNIQUE (email)
) ENGINE=InnoDB;

CREATE TABLE bookings (
    booking_id CHAR(36) NOT NULL PRIMARY KEY,
    customer_id CHAR(36) NOT NULL,
    show_id CHAR(36) NOT NULL,
    status ENUM('HELD', 'CONFIRMED', 'EXPIRED', 'CANCELLED') NOT NULL,
    created_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    expires_at DATETIME(6) NULL,
    CONSTRAINT chk_bookings_expiry CHECK (
        (status = 'HELD' AND expires_at IS NOT NULL)
        OR (status <> 'HELD' AND expires_at IS NULL)
    ),
    CONSTRAINT fk_bookings_customer FOREIGN KEY (customer_id)
        REFERENCES customers (customer_id),
```

```
    CONSTRAINT fk_bookings_show FOREIGN KEY (show_id)
        REFERENCES shows (show_id),
    INDEX ix_bookings_expiry (status, expires_at, booking_id),
    INDEX ix_bookings_customer_created (customer_id, created_at)
) ENGINE=InnoDB;

CREATE TABLE show_seats (
    show_id CHAR(36) NOT NULL,
    seat_id CHAR(36) NOT NULL,
    price_amount DECIMAL(12, 2) NOT NULL,
    state ENUM('AVAILABLE', 'HELD', 'BOOKED') NOT NULL DEFAULT 'AVAILABLE',
    hold_expires_at DATETIME(6) NULL,
    PRIMARY KEY (show_id, seat_id),
    CONSTRAINT chk_show_seats_price CHECK (price_amount >= 0),
    CONSTRAINT chk_show_seats_hold CHECK (
        (state = 'HELD' AND hold_expires_at IS NOT NULL)
        OR (state <> 'HELD' AND hold_expires_at IS NULL)
    ),
    CONSTRAINT fk_show_seats_show FOREIGN KEY (show_id)
        REFERENCES shows (show_id),
    CONSTRAINT fk_show_seats_seat FOREIGN KEY (seat_id)
        REFERENCES seats (seat_id),
    INDEX ix_show_seats_state (show_id, state, seat_id)
) ENGINE=InnoDB;

CREATE TABLE booking_seats (
    booking_id CHAR(36) NOT NULL,
    seat_id CHAR(36) NOT NULL,
    price_amount DECIMAL(12, 2) NOT NULL,
    PRIMARY KEY (booking_id, seat_id),
    CONSTRAINT chk_booking_seats_price CHECK (price_amount >= 0),
    CONSTRAINT fk_booking_seats_booking FOREIGN KEY (booking_id)
        REFERENCES bookings (booking_id),
    CONSTRAINT fk_booking_seats_seat FOREIGN KEY (seat_id)
        REFERENCES seats (seat_id),
    INDEX ix_booking_seats_seat (seat_id, booking_id)
) ENGINE=InnoDB;

CREATE TABLE payments (
    payment_id CHAR(36) NOT NULL PRIMARY KEY,
    booking_id CHAR(36) NOT NULL,
    provider VARCHAR(60) NOT NULL,
    provider_payment_id VARCHAR(190) NULL,
    idempotency_key VARCHAR(190) NOT NULL,
    amount DECIMAL(12, 2) NOT NULL,
    status ENUM('PENDING', 'SUCCEEDED', 'FAILED', 'REFUNDED') NOT NULL DEFAULT 'PENDING',
    created_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    updated_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    CONSTRAINT uq_payments_idempotency UNIQUE (provider, idempotency_key),
    CONSTRAINT uq_payments_provider_id UNIQUE (provider, provider_payment_id),
    CONSTRAINT chk_payments_amount CHECK (amount >= 0),
    CONSTRAINT fk_payments_booking FOREIGN KEY (booking_id)
        REFERENCES bookings (booking_id),
    INDEX ix_payments_booking_created (booking_id, created_at)
) ENGINE=InnoDB;

CREATE TABLE payment_webhook_events (
    provider VARCHAR(60) NOT NULL,
    provider_event_id VARCHAR(190) NOT NULL,
    payment_id CHAR(36) NULL,
    received_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    processed_at DATETIME(6) NULL,
    payload JSON NOT NULL,
    PRIMARY KEY (provider, provider_event_id),
    CONSTRAINT fk_webhook_payment FOREIGN KEY (payment_id)
        REFERENCES payments (payment_id),
    INDEX ix_webhook_payment (payment_id)
) ENGINE=InnoDB;

DELIMITER $$

-- Serialize schedule writes on the screen row, then reject overlapping intervals.
CREATE TRIGGER shows_no_overlap_before_insert
BEFORE INSERT ON shows
FOR EACH ROW
BEGIN
    DECLARE locked_screen CHAR(36);
    DECLARE overlapping_show CHAR(36) DEFAULT NULL;
    SELECT screen_id INTO locked_screen
    FROM screens
    WHERE screen_id = NEW.screen_id
    FOR UPDATE;

    IF NEW.status = 'SCHEDULED' THEN
                SELECT show_id INTO overlapping_show
        FROM shows
        WHERE screen_id = NEW.screen_id
          AND status = 'SCHEDULED'
```

```
          AND starts_at < NEW.ends_at
                    AND ends_at > NEW.starts_at
                LIMIT 1
                FOR UPDATE;
                IF overlapping_show IS NOT NULL THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Show overlaps another scheduled show on this
                screen';
        END IF;
    END IF;
END$$

CREATE TRIGGER shows_no_overlap_before_update
BEFORE UPDATE ON shows
FOR EACH ROW
BEGIN
    DECLARE locked_screen CHAR(36);
    DECLARE overlapping_show CHAR(36) DEFAULT NULL;
    SELECT screen_id INTO locked_screen
    FROM screens
    WHERE screen_id = NEW.screen_id
    FOR UPDATE;

    IF NEW.status = 'SCHEDULED' THEN
                SELECT show_id INTO overlapping_show
        FROM shows
        WHERE screen_id = NEW.screen_id
          AND show_id <> OLD.show_id
          AND status = 'SCHEDULED'
          AND starts_at < NEW.ends_at
                    AND ends_at > NEW.starts_at
                LIMIT 1
                FOR UPDATE;
                IF overlapping_show IS NOT NULL THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Show overlaps another scheduled show on this
                screen';
        END IF;
    END IF;
END$$

CREATE TRIGGER show_seats_screen_match_before_insert
BEFORE INSERT ON show_seats
FOR EACH ROW
BEGIN
    DECLARE matching_rows INT DEFAULT 0;
    SELECT COUNT(*) INTO matching_rows
    FROM shows AS sh
    JOIN seats AS st ON st.seat_id = NEW.seat_id
    WHERE sh.show_id = NEW.show_id
      AND sh.screen_id = st.screen_id;
    IF matching_rows <> 1 THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Seat does not belong to the screen for this show';
    END IF;
END$$

CREATE TRIGGER show_seats_screen_match_before_update
BEFORE UPDATE ON show_seats
FOR EACH ROW
BEGIN
    DECLARE matching_rows INT DEFAULT 0;
    SELECT COUNT(*) INTO matching_rows
    FROM shows AS sh
    JOIN seats AS st ON st.seat_id = NEW.seat_id
    WHERE sh.show_id = NEW.show_id
      AND sh.screen_id = st.screen_id;
    IF matching_rows <> 1 THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Seat does not belong to the screen for this show';
    END IF;
END$$

CREATE TRIGGER booking_seats_show_match_before_insert
BEFORE INSERT ON booking_seats
FOR EACH ROW
BEGIN
    DECLARE matching_rows INT DEFAULT 0;
    SELECT COUNT(*) INTO matching_rows
    FROM bookings AS b
    JOIN show_seats AS ss ON ss.show_id = b.show_id
    WHERE b.booking_id = NEW.booking_id
      AND ss.seat_id = NEW.seat_id;
    IF matching_rows <> 1 THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Seat has no inventory for this booking show';
    END IF;
END$$

CREATE TRIGGER booking_seats_show_match_before_update
BEFORE UPDATE ON booking_seats
FOR EACH ROW
BEGIN
```

```
    DECLARE matching_rows INT DEFAULT 0;
    SELECT COUNT(*) INTO matching_rows
    FROM bookings AS b
    JOIN show_seats AS ss ON ss.show_id = b.show_id
    WHERE b.booking_id = NEW.booking_id
      AND ss.seat_id = NEW.seat_id;
    IF matching_rows <> 1 THEN
SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Seat has no inventory for this booking show';
    END IF;
END$$

DELIMITER ;
```

Appendix B: Sample Rows (P1)

```
USE booking_engine;

INSERT INTO theatres (theatre_id, name, address, timezone) VALUES
('10000000-0000-0000-0000-000000000001', 'PVR Central', 'MG Road, Bengaluru', 'Asia/Kolkata')
ON DUPLICATE KEY UPDATE name = VALUES(name);

INSERT INTO screens (screen_id, theatre_id, name) VALUES
('20000000-0000-0000-0000-000000000001', '10000000-0000-0000-0000-000000000001', 'Screen 1')
ON DUPLICATE KEY UPDATE name = VALUES(name);

INSERT INTO seats (seat_id, screen_id, row_label, seat_number, seat_type) VALUES
('30000000-0000-0000-0000-000000000001', '20000000-0000-0000-0000-000000000001', 'A', 1,
    'STANDARD'),
('30000000-0000-0000-0000-000000000002', '20000000-0000-0000-0000-000000000001', 'A', 2,
    'STANDARD'),
('30000000-0000-0000-0000-000000000003', '20000000-0000-0000-0000-000000000001', 'B', 1,
    'PREMIUM'),
('30000000-0000-0000-0000-000000000004', '20000000-0000-0000-0000-000000000001', 'B', 2,
    'PREMIUM')
ON DUPLICATE KEY UPDATE seat_id = VALUES(seat_id);

INSERT INTO movies (movie_id, title, language, duration_mins, rating) VALUES
('40000000-0000-0000-0000-000000000001', 'The Last Monsoon', 'Kannada', 142, 'U/A'),
('40000000-0000-0000-0000-000000000002', 'Orbit Seven', 'English', 128, 'U/A')
ON DUPLICATE KEY UPDATE title = VALUES(title);

INSERT INTO shows (show_id, movie_id, screen_id, starts_at, ends_at, status, currency) VALUES
('50000000-0000-0000-0000-000000000001', '40000000-0000-0000-0000-000000000001',
    '20000000-0000-0000-0000-000000000001', '2026-10-15 03:30:00', '2026-10-15 06:10:00',
    'SCHEDULED', 'INR'),
('50000000-0000-0000-0000-000000000002', '40000000-0000-0000-0000-000000000002',
    '20000000-0000-0000-0000-000000000001', '2026-10-15 07:00:00', '2026-10-15 09:25:00',
    'SCHEDULED', 'INR'),
('50000000-0000-0000-0000-000000000003', '40000000-0000-0000-0000-000000000001',
    '20000000-0000-0000-0000-000000000001', '2026-10-16 03:30:00', '2026-10-16 06:10:00',
    'SCHEDULED', 'INR')
ON DUPLICATE KEY UPDATE starts_at = VALUES(starts_at);

INSERT INTO customers (customer_id, display_name, email) VALUES
('60000000-0000-0000-0000-000000000001', 'Sample Customer', 'sample.customer@example.test')
ON DUPLICATE KEY UPDATE display_name = VALUES(display_name);

INSERT INTO show_seats (show_id, seat_id, price_amount)
SELECT sh.show_id, st.seat_id,
       CASE WHEN st.seat_type = 'PREMIUM' THEN 350.00 ELSE 250.00 END
FROM shows AS sh
JOIN seats AS st ON st.screen_id = sh.screen_id
WHERE sh.show_id IN (
    '50000000-0000-0000-0000-000000000001',
    '50000000-0000-0000-0000-000000000002',
    '50000000-0000-0000-0000-000000000003'
)
ON DUPLICATE KEY UPDATE price_amount = VALUES(price_amount);

INSERT INTO bookings (booking_id, customer_id, show_id, status, created_at, expires_at) VALUES
('70000000-0000-0000-0000-000000000001', '60000000-0000-0000-0000-000000000001',
    '50000000-0000-0000-0000-000000000001', 'CONFIRMED', '2026-10-14 10:00:00.000000', NULL)
ON DUPLICATE KEY UPDATE status = VALUES(status), expires_at = VALUES(expires_at);

INSERT INTO booking_seats (booking_id, seat_id, price_amount) VALUES
('70000000-0000-0000-0000-000000000001', '30000000-0000-0000-0000-000000000001', 250.00)
ON DUPLICATE KEY UPDATE price_amount = VALUES(price_amount);

UPDATE show_seats
SET state = 'BOOKED', hold_expires_at = NULL
WHERE show_id = '50000000-0000-0000-0000-000000000001'
  AND seat_id = '30000000-0000-0000-0000-000000000001';

INSERT INTO payments
(payment_id, booking_id, provider, provider_payment_id, idempotency_key, amount, status,
        created_at, updated_at)
VALUES
('80000000-0000-0000-0000-000000000001', '70000000-0000-0000-0000-000000000001', 'demo-pay',
        'demo-payment-001', 'demo-booking-001', 250.00, 'SUCCEEDED', '2026-10-14
        10:01:00.000000', '2026-10-14 10:01:00.000000')
ON DUPLICATE KEY UPDATE status = VALUES(status), amount = VALUES(amount);

INSERT INTO payment_webhook_events
    (provider, provider_event_id, payment_id, received_at, processed_at, payload)
VALUES
('demo-pay', 'evt-demo-001', '80000000-0000-0000-0000-000000000001', '2026-10-14
        10:01:01.000000', '2026-10-14 10:01:01.000000', JSON_OBJECT('type',
        'payment.succeeded', 'payment_id', 'demo-payment-001'))
ON DUPLICATE KEY UPDATE processed_at = VALUES(processed_at);
```

Appendix C: P2 and Operational SQL

```
USE booking_engine;

-- P2: list scheduled shows at one theatre for a theatre-local calendar date.
-- Set @theatre_id and @local_date before running. Requires MySQL time-zone tables.
SET @theatre_id = '10000000-0000-0000-0000-000000000001';
SET @local_date = DATE '2026-10-15';

SELECT
    sh.show_id,
    m.title AS movie_title,
    sc.name AS screen_name,
    CONVERT_TZ(sh.starts_at, '+00:00', th.timezone) AS starts_at_local,
    CONVERT_TZ(sh.ends_at, '+00:00', th.timezone) AS ends_at_local,
    sh.currency,
    SUM(CASE WHEN ss.state = 'AVAILABLE' THEN 1 ELSE 0 END) AS available_seats
FROM theatres AS th
JOIN screens AS sc ON sc.theatre_id = th.theatre_id
JOIN shows AS sh ON sh.screen_id = sc.screen_id
JOIN movies AS m ON m.movie_id = sh.movie_id
LEFT JOIN show_seats AS ss ON ss.show_id = sh.show_id
WHERE th.theatre_id = @theatre_id
  AND sh.status = 'SCHEDULED'
  AND @local_date >= DATE(CONVERT_TZ(UTC_TIMESTAMP(), '+00:00', th.timezone))
AND @local_date < DATE_ADD(DATE(CONVERT_TZ(UTC_TIMESTAMP(), '+00:00', th.timezone)), INTERVAL 7
      DAY)
  AND sh.starts_at >= CONVERT_TZ(CONCAT(@local_date, ' 00:00:00'), th.timezone, '+00:00')
AND sh.starts_at < CONVERT_TZ(CONCAT(DATE_ADD(@local_date, INTERVAL 1 DAY), ' 00:00:00'),
      th.timezone, '+00:00')
GROUP BY sh.show_id, m.title, sc.name, sh.starts_at, sh.ends_at, th.timezone, sh.currency
ORDER BY sh.starts_at, sc.name;

-- P1 seat hold: run in one transaction. Bind the selected IDs as parameters.
-- The application must compare the returned row count with the unique seat count;
-- rollback if any requested seat is missing. Insert booking + booking_seats and
-- update inventory before COMMIT. Example lock:
-- SELECT seat_id, price_amount
-- FROM show_seats
-- WHERE show_id = ? AND seat_id IN (?, ?) AND state = 'AVAILABLE'
-- ORDER BY seat_id
-- FOR UPDATE SKIP LOCKED;

-- Expiry worker claim (execute in a transaction; workers skip already-claimed holds).
SELECT booking_id, show_id
FROM bookings
WHERE status = 'HELD' AND expires_at <= UTC_TIMESTAMP(6)
ORDER BY expires_at, booking_id
LIMIT 100
FOR UPDATE SKIP LOCKED;

-- Webhook idempotency gate. A duplicate-key error means this event was already applied.
-- Catch MySQL error 1062 and acknowledge the duplicate without repeating effects.
INSERT INTO payment_webhook_events (provider, provider_event_id, payment_id, payload)
VALUES ('example-provider', 'evt-example-001', NULL, JSON_OBJECT('type', 'payment.succeeded'));
```